Estado del arte de los paradigmas de programación (2025–2026)

Los paradigmas de programación han evolucionado desde un enfoque de "elección única" hacia una integración pragmática y multiparadigma, donde lenguajes modernos combinan programación funcional, reactiva, orientada a objetos y declarativa según la capa arquitectónica y el problema específico. Paralelamente, emergen nuevos paradigmas ligados a hardware no convencional (neuromórfico, cuántico, computación en memoria) que exigen repensar abstracciones clásicas.[1] [2] [3] [4] [5]

1. Panorama general: tendencias dominantes

- **Multiparadigma pragmático**: Para 2025–2026, la tendencia no es "FP vs OOP", sino cómo combinarlos: capas de dominio con OOP, lógica de negocio con FP, manejo de eventos con programación reactiva y consultas con enfoques declarativos (p. ej., SQL).[2] [1]
- **Ascenso de lo funcional y reactivo**: La programación funcional (inmutabilidad, funciones puras, composición) gana terreno en ingeniería de datos, IA y arquitecturas cloud-native; la programación reactiva se consolida para flujos asíncronos, alta concurrencia y sistemas en tiempo real.[6] [7] [1]
- **Declaratividad y especialización**: Paradigmas declarativos (lógica, restricciones, consultas) y enfoques orientados a IA (aprendizaje automático como "programación automatizada") se integran en pipelines y sistemas de decisión.[5] [2]

2. Paradigmas clásicos: estado actual y evolución

Programación orientada a objetos (OOP)

- **Fortalezas vigentes**: Modelado de entidades con identidad y ciclo de vida (dominios ricos, sistemas empresariales, motores de juego). Encapsulamiento, polimorfismo y abstracción siguen siendo pilares arquitectónicos.[1]
- **Limitaciones reconocidas**: Acoplamiento por herencia, estado mutable compartido (complejidad en concurrencia) y boilerplate. La práctica moderna favorece composición sobre herencia y combina OOP con FP para reducir rigidez.[1]

Programación funcional (FP)

- **Principios clave**: Inmutabilidad, funciones puras, funciones de primera clase y estilo declarativo. Esto facilita razonamiento, testing y paralelismo.[7] [1]
- **Ámbitos de impacto 2025–2026**: Pipelines de transformación de datos, procesamiento de streams, lógica de negocio compleja y sistemas distribuidos donde la ausencia de estado mutable compartido simplifica la concurrencia.[6] [1]

-

Programación reactiva

- **Modelo mental**: Flujos de datos en el tiempo, composición asíncrona no bloqueante y backpressure para evitar saturación.[1]
- **Casos de uso**: Microservicios de alta concurrencia, analítica en tiempo real, interfaces reactivas y agregación de sensores IoT. No es ideal para CRUD simples o cargas CPU-bound con APIs bloqueantes.[1]

Programación lógica y por restricciones

- **Enfoque**: Especificar "qué" debe cumplirse (hechos y reglas) y dejar que el motor de inferencia busque soluciones (p. ej., Prolog, CLP).[1]
- **Aplicaciones**: Sistemas expertos, NLP simbólico, planificación y problemas de satisfacción de restricciones. El rendimiento depende de heurísticas y la depuración requiere entender la estrategia de búsqueda.[1]

3. Paradigmas emergentes ligados a nuevo hardware

Programación neuromórfica

- **Motivación**: Hardware inspirado en el cerebro (spiking neural networks, eventos, tiempo continuo, plasticidad) no encaja bien con supuestos clásicos (digital, determinista, centralizado, observable).[4] [5]
- **Características fundamentales**:
    - **Dominio físico y tiempo continuo** (señales analógicas/mixtas),
    - **Plasticidad** (el sustrato cambia durante la ejecución),
    - **Estocasticidad**,
    - **Descentralización**,
    - **Observabilidad limitada**.[5]
- **Enfoques actuales**:
    - **Co-diseño hardware-so ware**,
    - **Métodos de ML adaptados a SNN** (backprop con gradientes sustitutos, conversión
    - ANN→SNN),
    - **Aprendizaje en dispositivo** (plasticidad local, reglas de aprendizaje emergentes),
    - **Métodos evolutivos** (optimizan arquitectura y pesos conjuntamente),
    - **Algoritmos neuromórficos de picos (SNA)** y **primitivas neurocomputacionales** (WTA, campos neuronales dinámicos, osciladores),
    - **Abstracciones de mayor nivel** (Neural Engineering Framework, Vector Symbolic
    - Architectures, frameworks como Lava y representaciones intermedias como NIR).[5]
- **Desafío abierto**: No existe aún una abstracción universalmente aceptada; se avanza hacia representaciones intermedias que integren múltiples enfoques y soporten grafos computacionales que cambian en el tiempo.[5]

-

Programación cuántica

- **Estado**: Lenguajes y runtimes cuánticos avanzan hacia integración con HPC, pero persisten retos en la definición del QPU y su gestión de tareas asíncronas en entornos científicos.[8] [9]
- **Paradigmas**: Se exploran descripciones de alto nivel de algoritmos cuánticos, composición híbrida clásico-cuántica y modelos de ejecución compatibles con flujos HPC.[9] [8]

Otros paradigmas emergentes

- **Computación en memoria y arquitecturas especializadas**: Requieren abstracciones que expongan operaciones near-data y minimicen movimientos de datos.[3]
- **Paradigmas "no digitales"**: Programación analógica, computación de reservorio físico y "evolution in materio" aprovechan dinámicas del sustrato para computar.[5]

4. Integración multiparadigma: patrones arquitectónicos actuales

Una arquitectura típica en 2025–2026 combina paradigmas por capa:

- **Dominio**: OOP para entidades con identidad (Usuario, Orden, Producto).[1]
- **Lógica de negocio**: FP para transformaciones sin estado (validación, reglas, cálculos).[1]
- **Eventos/Streams**: Programación reactiva para actualizaciones en tiempo real y backpressure.[1]
- **Datos**: Consultas declarativas (SQL, lenguajes de restricciones) para especificar "qué" recuperar.[1]
- **Integración**: Mensajería asíncrona entre contextos delimitados.[1]

Esto refleja un cambio cultural: elegir el paradigma según el problema, no por lealtad ideológica.[2] [1]

-

Referencias:

1. https://www.javacodegeeks.com/2026/01/paradigm-shi s-in-programming-languages-object-oriented-functional-and -beyond.html
2. https://usajournals.org/index.php/2/article/view/885
3. https://www.mdpi.com/2079-9292/14/12/2476
4. https://ieeexplore.ieee.org/document/10766507/
5. https://arxiv.org/html/2410.22352v1
6. https://blog.madrigan.com/en/blog/202510141136/
7. https://blog.madrigan.com/en/blog/202604110953/
8. https://ieeexplore.ieee.org/document/10566297/
9. https://link.springer.com/10.1007/978-3-031-61763-8_8
10. https://www.sciencedirect.com/science/article/pii/S0925231226019788
11. https://www.nature.com/articles/s44335-026-00064-6
12. https://link.springer.com/book/10.1007/978-3-031-99751-8
13. https://ieeexplore.ieee.org/document/11633693/
14. https://arxiv.org/pdf/2310.18260.pdf
15. https://ieeexplore.ieee.org/document/11275564/
16. https://www.igi-global.com/chapter/quantum-neuromorphic-intelligence/414321
17. http://oa.upm.es/82008/
18. https://wjarr.com/node/7395
19. https://www.worldscientific.com/doi/10.1142/S0217979226300112
20. https://ieeexplore.ieee.org/document/11556079/
21. https://nerdleveltech.com/programming-paradigms-compared-from-oop-to-functional-thinking
22. https://artybondar.github.io/blog/programming-2026.html
23. https://eajournals.org/ejcsit/wp-content/uploads/sites/21/2025/05/Reactive-Programming.pdf
24. https://adabeat.com/fp/will-functional-programming-grow-in-2025/
25. https://entangledfuture.com/learn/quantum-vs-neuromorphic-computing/

Nota: Elaborado con perplexity.